AI-Powered Document Insights and Data Extraction

Name: Kian Ranjbar
Email: keyonranjbar@aol.com

Problem Statement | To design a tool that assist you in research. Something to help you organize disheveled documents and gain insight from it. A research assistant.

Challenge

When it comes to reading medical studies, it can often get overwhelming. Sometimes they are hard to comprehend, sometimes when you are studying a topic, you have multiple sources of research to sift through. We need a way of organizing and viewing this information clear and concisely. Also being able to converse with documents and ask it questions would really elevate the design.

Solution

I developed an AI chatbot, with a Gradio interface. It cleans documents to be scanned by OCR, extracts the important data, then using AI models to understand the contents. The RAG (Retrieval-Augmented Generation) system allows users to ask questions.

Pain Point	How Your Pipeline Solves It
Disheveled documents	Organizes them
Difficult to understand	Able to summarize coherently
I have questions about the document	The chatbot will allow you to ask any question to that specific document
I have multiple different files pertaining to the same topic	I can organize and summarize this all, you can ask questions about any part of any document

System Architecture | AI Gradio Research Assistant Chatbot

[My Pipeline Flow:]
Blob PDF → Page Splitting & Classification → Metadata Tagging (doc, type, page number) → OCR + Text Cleaning → Chunking (semantic + fixed) → Embedding Modeling → Vector Storage → Retriever + Reranker → LLM with Context Prompt → Final Answers

Component Specifications

Component	Technology Choice	Configuration Details
OCR Engine	Tesseract OCR via Python pytesseract	Uses pytesseract.image_to_string() for scanned PDFs when normal text extraction fails. Installed with tesseract-ocr. OCR is performed page-by-page after converting PDF pages to PNG images using PyMuPDF. No custom language packs specified, so default English OCR is likely being used.
Text Chunking	Custom fixed-size overlapping chunking + optional semantic chunking with LlamaIndex	Default chunk size: 500 words. Overlap: 100 words. Uses sliding window chunking in chunk_document_with_metadata(). Optional SentenceSplitter from LlamaIndex enables sentence-aware chunking with paragraph preservation. Metadata such as document type, page range, and chunk ID are preserved.
Embeddings	Sentence Transformers using all-MiniLM-L6-v2	Embedding model: sentence-transformers/all-MiniLM-L6-v2. Approx. 384-dimensional vectors. Lightweight and optimized for semantic similarity search. Used both through SentenceTransformer and HuggingFaceEmbedding for compatibility with LlamaIndex.
Vector Database	FAISS	Uses faiss.IndexFlatL2 for dense vector similarity search. Similarity metric: L2 distance. In-memory vector storage only (not persistent). Separate FAISS indices are created for each detected document type for intelligent query routing.
Retriever	Dense semantic retrieval with intelligent query routing	Retrieves top-K chunks (k=4 default, adjustable 1–10 in UI). Uses embedding similarity search through FAISS. Includes automatic query-to-document-type routing using the LLM. Supports filtered retrieval by document type and confidence-based routing fallback to global search.
LLM	Mistral 7B Instruct running locally via llama-cpp-python	Model: mistral-7b-instruct-v0.2.Q4_K_M.gguf. Loaded locally with GPU acceleration (n_gpu_layers=1). Context window: 2048. Temperature varies by task (0.0–0.3). Max generation tokens typically 128–512. Used for classification, boundary detection, query routing, and answer generation.
Prompt Strategy	Context-grounded RAG prompting with instruction-based prompting	Uses structured prompts for document classification, boundary detection, retrieval routing, and QA generation. Answers are constrained to provided context only. Source attribution included in prompts. Context is truncated to ~4000 characters to stay within token limits.

Pipeline in Action | Testing with a random blob file

Example Query 1: Lets see how it handles organizing the data, and retrieving simple data from disparate documents. As well as handling multiple questions at the same time.

Query: "What is the title of the first page? And tell me what type of document is page 4?"

Retrieved Chunks:

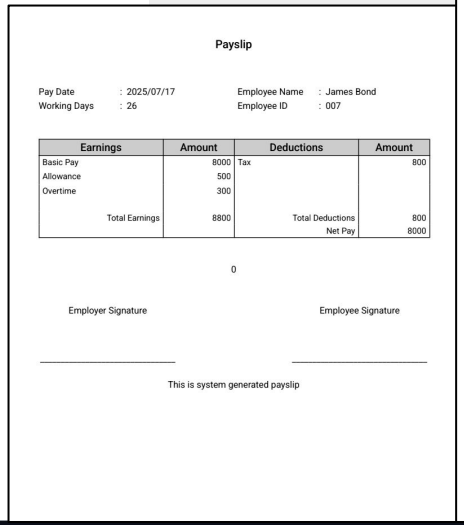
- *Other (Pages 1-1) - Relevance: 21.93%*
- *Other (Pages 5-5) - Relevance: 4.31%*
- *Report (Pages 4-4) - Relevance: 1.45%*
- *Other (Pages 6-6) - Relevance: 0.00%*

Confidence: 6.9% | Filter: auto

Final Answer: "The total earnings on page 2 is 8800.

Page 4 is a contract or agreement document."

As you can see the chatbot found the correct amount, and described what type of document page 4 is, so it handled the query excellently.



Payslip

Pay Date : 2025/07/17
Working Days : 26
Employee Name : James Bond
Employee ID : 007

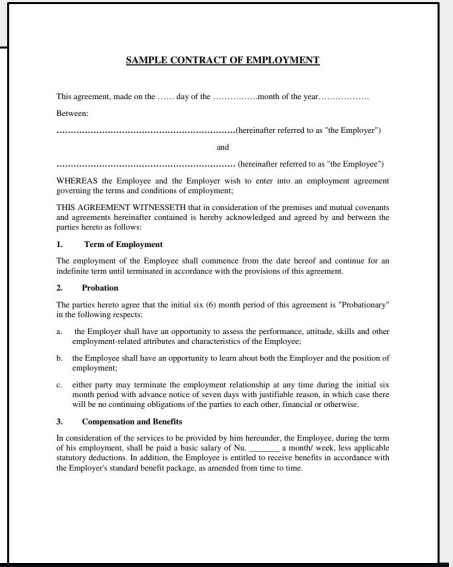
Earnings	Amount	Deductions	Amount
Basic Pay	8000	Tax	800
Allowance	500		
Overtime	300		
Total Earnings	8800	Total Deductions	800
		Net Pay	8000

0

Employer Signature

Employee Signature

This is system generated payslip



SAMPLE CONTRACT OF EMPLOYMENT

This agreement, made on the …… day of the ……..month of the year……

Between:

……(hereinafter referred to as "the Employer")

and

…… (hereinafter referred to as "the Employee")

WHEREAS the Employee and the Employer wish to enter into an employment agreement governing the terms and conditions of employment;

THIS AGREEMENT WITNESSETH that in consideration of the premises and mutual covenants and agreements hereinafter contained is hereby acknowledged and agreed by and between the parties hereto as follows:

1. Term of Employment

The employment of the Employee shall commence from the date hereof and continue for an indefinite term until terminated in accordance with the provisions of this agreement.

2. Probation

The parties hereto agree that the initial six (6) month period of this agreement is "Probationary" in the following respects:

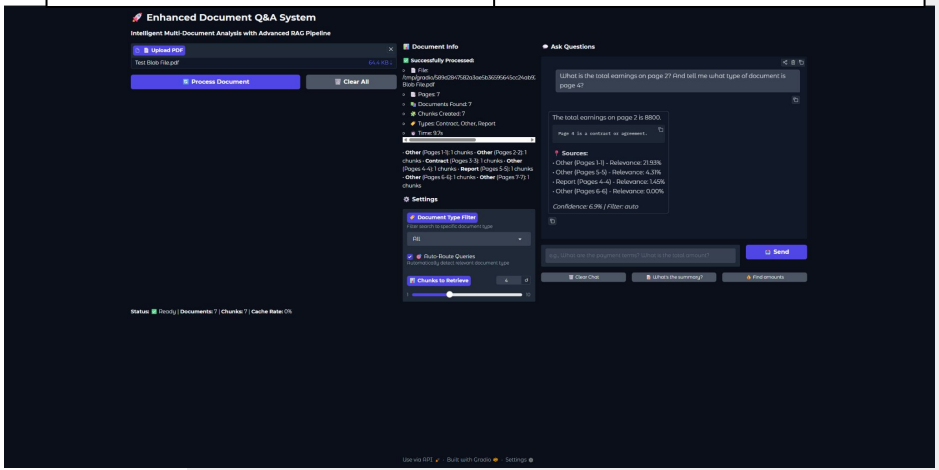
a. the Employer shall have an opportunity to assess the performance, attitude, skills and other employment-related attributes and characteristics of the Employee;

b. the Employee shall have an opportunity to learn about both the Employer and the position of employment;

c. either party may terminate the employment relationship at any time during the initial six month period with advance notice of seven days with justifiable reason, in which case there will be no continuing obligations of the parties to each other, financial or otherwise.

3. Compensation and Benefits

In consideration of the services to be provided by him hereunder, the Employee, during the term of his employment, shall be paid a basic salary of Nu. _____ a month/ week, less applicable statutory deductions. In addition, the Employee is entitled to receive benefits in accordance with the Employer's standard benefit package, as amended from time to time.



Design Decision Analysis | The key to this design was fast execution on Google Collab platform, while using minimal resources

Model Selection Reasoning

Decision	Rationale	Trade-offs Considered
Embedding Model Choice — all-MiniLM-L6-v2	Because it is lightweight, fast, free, and performs well for semantic similarity tasks in RAG systems. It generates strong embeddings while requiring relatively low compute resources, making it ideal for local execution in Google Colab. The model also integrates smoothly with both SentenceTransformers and LlamaIndex.	Sacrificed some semantic accuracy and multilingual capability compared to larger embedding models like BGE-large or OpenAI embeddings. Higher-end models could improve retrieval quality but would require more VRAM, inference time, or API costs.
Chunking Strategy — Fixed-size overlapping chunks with optional semantic chunking	Fixed-size chunking with overlap was selected because it is simple, predictable, and preserves contextual continuity between chunks. The overlap reduces the risk of losing important information split across chunk boundaries. Optional LlamaIndex sentence-based chunking was included for improved semantic coherence.	More advanced semantic or recursive chunking methods may preserve meaning better, but they increase processing complexity and runtime. Smaller chunks improve retrieval precision but risk losing context, while larger chunks increase context retention but reduce retrieval granularity.
LLM Choice — Mistral 7B Instruct via llama-cpp-python	Chosen because it offers strong instruction-following performance while remaining lightweight enough to run locally. Running the model locally eliminates API costs, improves privacy for sensitive documents like mortgage files, and avoids external dependency issues. Quantized GGUF format also reduces memory requirements significantly.	Sacrificed some reasoning quality and context length compared to larger cloud models like GPT-4 or Claude. Local inference is slower than hosted APIs on powerful infrastructure, and the 2048-token context window limits how much retrieved information can be processed at once.
Vector DB Choice — FAISS	Selected because FAISS is fast, lightweight, open-source, and highly optimized for dense vector similarity search. It works extremely well for local prototypes and research projects without requiring external infrastructure. Creating separate indices per document type also improved retrieval efficiency.	FAISS lacks built-in persistence, distributed scaling, metadata filtering sophistication, and cloud deployment features available in databases like Pinecone or Weaviate. The system remains memory-based and less production-ready for very large-scale enterprise deployments.

Current Limitations & Next Steps

Current Limitations

1. **Input Processing Issues**
    - The OCR pipeline struggles with low-quality scans, rotated pages, handwritten text, and heavily compressed PDFs. For example, mortgage documents with blurry signatures or skewed scanned bank statements may produce inaccurate text extraction results.
2. **Retrieval Gaps**
    - The system can struggle with highly ambiguous or extremely broad questions such as “Summarize everything important in this mortgage file” because relevant information may span many chunks across multiple document types.

Proposed Enhancements

Short-term ([Timeframe - e.g., Next 2 weeks]):

- Enhance the interface, customize and make it look more appealing

Medium-term ([Timeframe - e.g., Next month]):

- Host it on the internet so anyone can use it anywhere

Long-term Vision:

- Make it so that it can seamlessly integrate with a database of medical literature and studies

Project Impact & Learning Outcomes | Attained skills in developing end-to-end RAG pipelines using Python

🧪 Key Technical Learnings

- **Retrieval-Augmented Generation (RAG) Architecture:** Learned how modern RAG pipelines combine embeddings, vector search, chunking, and LLM reasoning to enable intelligent question answering over unstructured documents. Gained practical experience with semantic retrieval, metadata-aware retrieval, query routing, and source-grounded answer generation.
- **OCR, Embeddings, and Vector Search:** Developed hands-on experience using Tesseract OCR for scanned document extraction, Sentence Transformers for semantic embeddings, and FAISS for high-speed vector similarity search. Also learned how document chunking strategies directly affect retrieval quality and response accuracy.
- **Multi-Document Intelligence Challenges:** Overcame challenges involving messy mortgage “blob” files containing multiple unrelated document types merged into a single PDF. Implemented document boundary detection, automatic document classification, metadata preservation, and query routing to improve retrieval precision and reduce irrelevant context during answer generation.

💊 Business Impact Potential

- **Efficiency Gains:** The system can reduce document review and search time from hours to seconds by automatically classifying, chunking, indexing, and retrieving relevant information from large document collections.
- **Accuracy Improvement:** Semantic retrieval combined with document-type routing improves retrieval precision compared to basic keyword search systems.
- **Scalability:** The architecture is modular and scalable for enterprise document intelligence systems. The pipeline could be expanded to process thousands of PDFs using cloud-hosted vector databases, larger LLMs, distributed OCR pipelines, and API-based inference services.

💇 Skills Developed

- Built an end-to-end RAG pipeline using Python, FAISS, semantic embeddings, OCR, and local LLM inference.
- Developed experience with frameworks and tools including LlamaIndex, Gradio, FAISS, Sentence Transformers, and Mistral 7B Instruct.
- Strengthened problem-solving and systems-design skills by debugging OCR failures, optimizing retrieval quality, balancing performance vs. compute limitations, and designing a user-friendly document intelligence interface.

Thank you